MICRO-ROVER (ZADE)

Build a WiFi-Controlled Robot Powered by a Pocket Power Bank

A Research & Build Documentation

Platform: ESP8266 | Language: C++ (Arduino) | Control: Web Browser Repository: github.com/RemanDey/micro-rover | License: MIT

Abstract

This document presents the design, implementation, and performance analysis of Micro-Rover (Zade) — a fully battery-operated, WiFi-controlled differential-drive robot built on the ESP8266 microcontroller. The system serves a self-hosted web dashboard from flash memory, enabling control from any modern browser without a dedicated app or radio transmitter. Four distinct control modalities are implemented: virtual joystick, keyboard input, smartphone tilt (accelerometer), and a novel Draw Mode in which the user sketches a trajectory that the rover autonomously replicates. The rover is powered exclusively by a standard 5V USB power bank, demonstrating that capable mobile robotics can be achieved with minimal, commodity hardware. This paper covers hardware architecture, firmware design, wireless communication, power modeling, differential-drive kinematics, and an open REST API for programmatic control.

Key Contributions

- Self-hosted web UI served from ESP8266 flash — no cloud, no app, no router required
- Novel Draw Mode: user-sketched paths replayed as real-world motor sequences
- Dual WiFi (AP + STA) simultaneous operation with mDNS discovery
- Full battery-power model: runtime estimation across common power bank sizes
- Open REST API: /move?left=X&right=Y accepting ±1023 signed PWM values

1. Introduction

Low-cost WiFi-enabled microcontrollers have dramatically reduced the barrier to entry for hobbyist robotics. Where earlier robot platforms required dedicated RF transceivers, custom Android applications, or proprietary radio protocols, the ESP8266 System-on-Chip (SoC) integrates a complete 802.11 b/g/n radio, TCP/IP stack, and 32-bit Tensilica LX106 processor into a component costing under $3 USD. This convergence opens an important design question: how much robotic capability — sensing, actuation, communication, user interface — can be delivered from a single-chip platform running entirely on a commodity USB power bank?

Micro-Rover (Zade) addresses this question directly. The project achieves browser-based control with trajectory visualization, multiple input modalities, and an autonomous path-following feature, all while consuming under 500 mA at peak load — well within the 1A–2.4A output range of even the smallest power banks on the market. This document serves as both a build guide for the Instructables community and a technical reference for researchers and students interested in embedded web servers, differential drive kinematics, and battery-constrained IoT design.

2. System Architecture

The Micro-Rover system consists of three tightly integrated layers: the hardware layer (chassis, motors, motor driver), the firmware layer (ESP8266 running Arduino C++), and the user interface layer (browser-based web dashboard). Figure 1 illustrates the complete system block diagram.

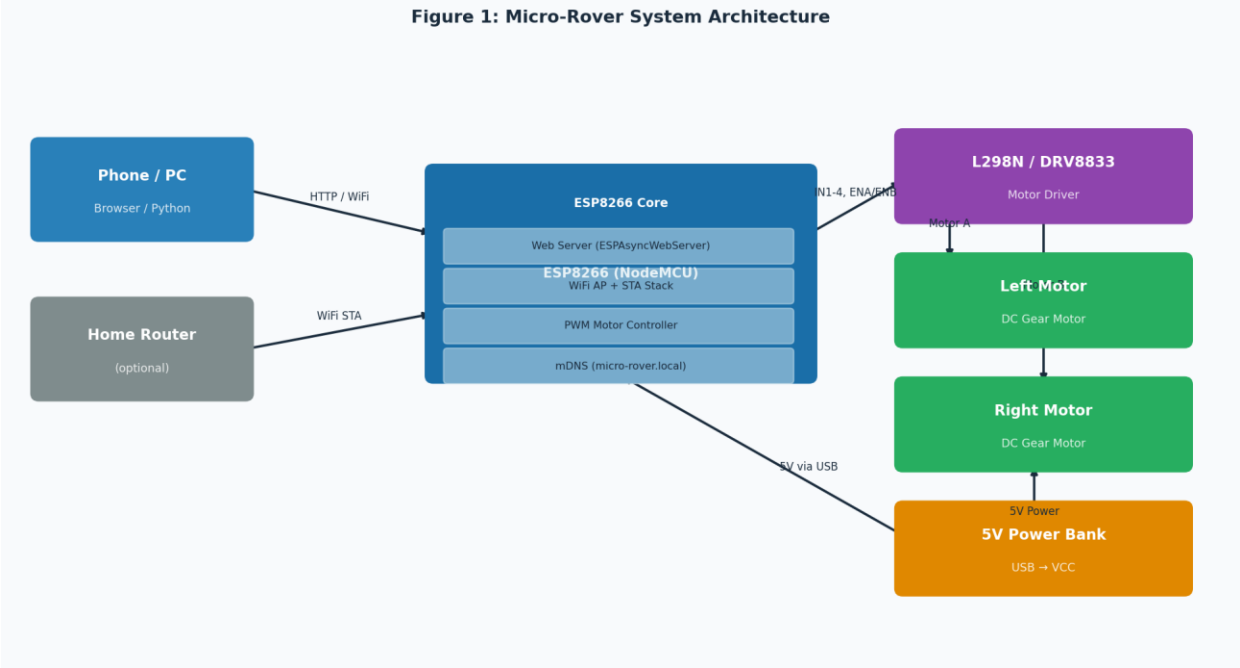

Figure 1: Complete System Architecture Block Diagram

2.1 Hardware Layer

The rover uses a 2WD differential-drive chassis — two independently driven rear wheels plus a passive front caster. Differential steering is achieved by varying the relative speed of the left and right motors; no servo or steering mechanism is required. This topology is mechanically simple, highly reliable, and well-suited to low-power operation.

The motor driver (L298N, L293D, or DRV8833) acts as a bridge between the ESP8266's 3.3V logic signals and the 5V motor supply. The ESP8266's GPIO pins cannot source sufficient current to directly drive DC motors; the motor driver amplifies the control signals to the ampere-level currents motors require.

> Note on Motor Driver Selection:
> The L298N is the most common choice and uses an H-bridge topology based on bipolar transistors.
> Its voltage drop (~2V) reduces effective motor voltage. The DRV8833 uses MOSFETs with much lower
> Rds(on), improving efficiency — recommended for maximum battery runtime.

2.2 Firmware Layer

The firmware, written in Arduino C++ for the ESP8266, performs three core functions simultaneously: (1) maintaining WiFi connectivity in both AP and STA modes, (2) serving the compressed web dashboard over HTTP, and (3) processing motor control commands received via the /move REST endpoint. The ESPAsyncWebServer library handles HTTP requests asynchronously, ensuring the system remains responsive even under continuous high-frequency control input.

2.3 User Interface Layer

The web dashboard is a single-page HTML5 application compressed and stored in the ESP8266's program flash (PROGMEM). It renders a virtual joystick using the HTML5 Canvas API, handles keyboard input via JavaScript event listeners, reads smartphone orientation from the DeviceOrientation API for tilt control, and implements Draw Mode by recording and replaying motor command sequences. All communication to the rover uses standard HTTP GET requests, making the system compatible with any browser without WebSocket overhead.

3. Hardware Design & Wiring

Figure 2 details the complete wiring diagram connecting the ESP8266 to the motor driver and motors. Table 1 provides the pin mapping reference.

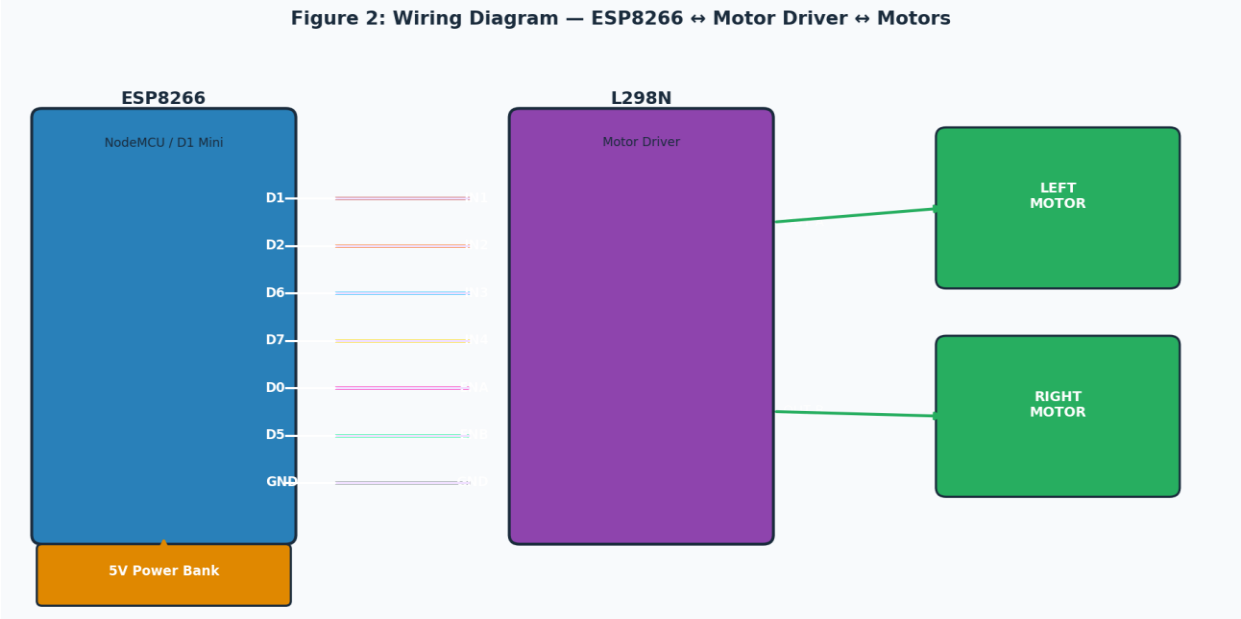

Figure 2: Wiring Diagram — ESP8266 to Motor Driver to Motors

Table 1: Pin Assignment Reference

ESP8266 Pin	Motor Driver Pin	Function	Signal Type
D1 (GPIO5)	IN1	Left Motor Forward	Digital OUT
D2 (GPIO4)	IN2	Left Motor Backward	Digital OUT
D6 (GPIO12)	IN3	Right Motor Forward	Digital OUT
D7 (GPIO13)	IN4	Right Motor Backward	Digital OUT
D0 (GPIO16)	ENA	Left Motor Speed	PWM (0–1023)
D5 (GPIO14)	ENB	Right Motor Speed	PWM (0–1023)
GND	GND	Common Ground	Reference

3.1 Power Architecture

A single 5V USB power bank feeds the system. The ESP8266 receives power through its Micro-USB port. The motor driver is powered from the same or a second USB output; on the L298N, an onboard 5V regulator can supply the ESP8266's VIN pin if preferred. The critical constraint is the common GND connection: without it, the motor driver's control inputs float relative to the ESP8266's logic level, causing erratic behavior.

> Critical Wiring Rule: Always connect GND between the ESP8266 and motor driver. Never power DC motors directly from the ESP8266's 3.3V or 5V pins — the inductive back-EMF of motors can damage the microcontroller. Always use a dedicated motor driver.

4. Wireless Communication Architecture

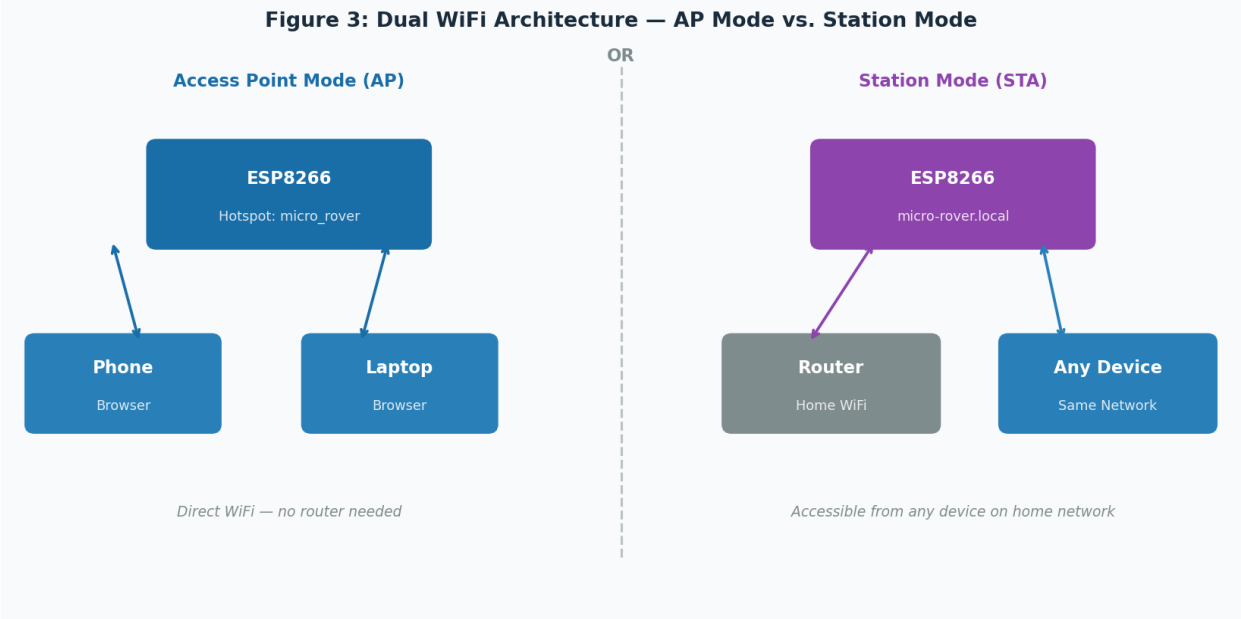

Figure 3: Dual WiFi Architecture — AP Mode (left) vs. Station Mode (right)

The ESP8266 supports simultaneous AP+STA operation: it broadcasts its own hotspot (micro_rover, password: 12345678) while also connecting to the user's home router if credentials are configured. This dual-mode design solves the deployment dilemma most IoT projects face — the rover works in any environment, even with no existing WiFi infrastructure.

mDNS (Multicast DNS) support via the ESP8266mDNS library allows users to navigate to http://micro-rover.local rather than memorizing a numeric IP address. mDNS is resolved locally by the OS without any DNS server, making it robust even on isolated networks.

Table 2: WiFi Mode Comparison

Property	Access Point (AP) Mode	Station (STA) Mode
Infrastructure needed	None — rover is the router	Existing WiFi router
Typical range	~15–20 m line of sight	Depends on home router
mDNS URL	http://micro-rover.local	http://micro-rover.local
Simultaneous clients	Up to 8 devices	N/A (rover is a client)
Latency (typical)	~35–50 ms	~40–60 ms
Use case	Field, outdoor, demo	Home, lab, multi-device

5. Firmware Design & Control Flow

Figure 6 shows the firmware execution flow from boot to steady-state operation. The system initialises WiFi and the web server once, then enters an event-driven loop handled entirely by the ESPAsyncWebServer interrupt model — there is no blocking delay() in the main loop.

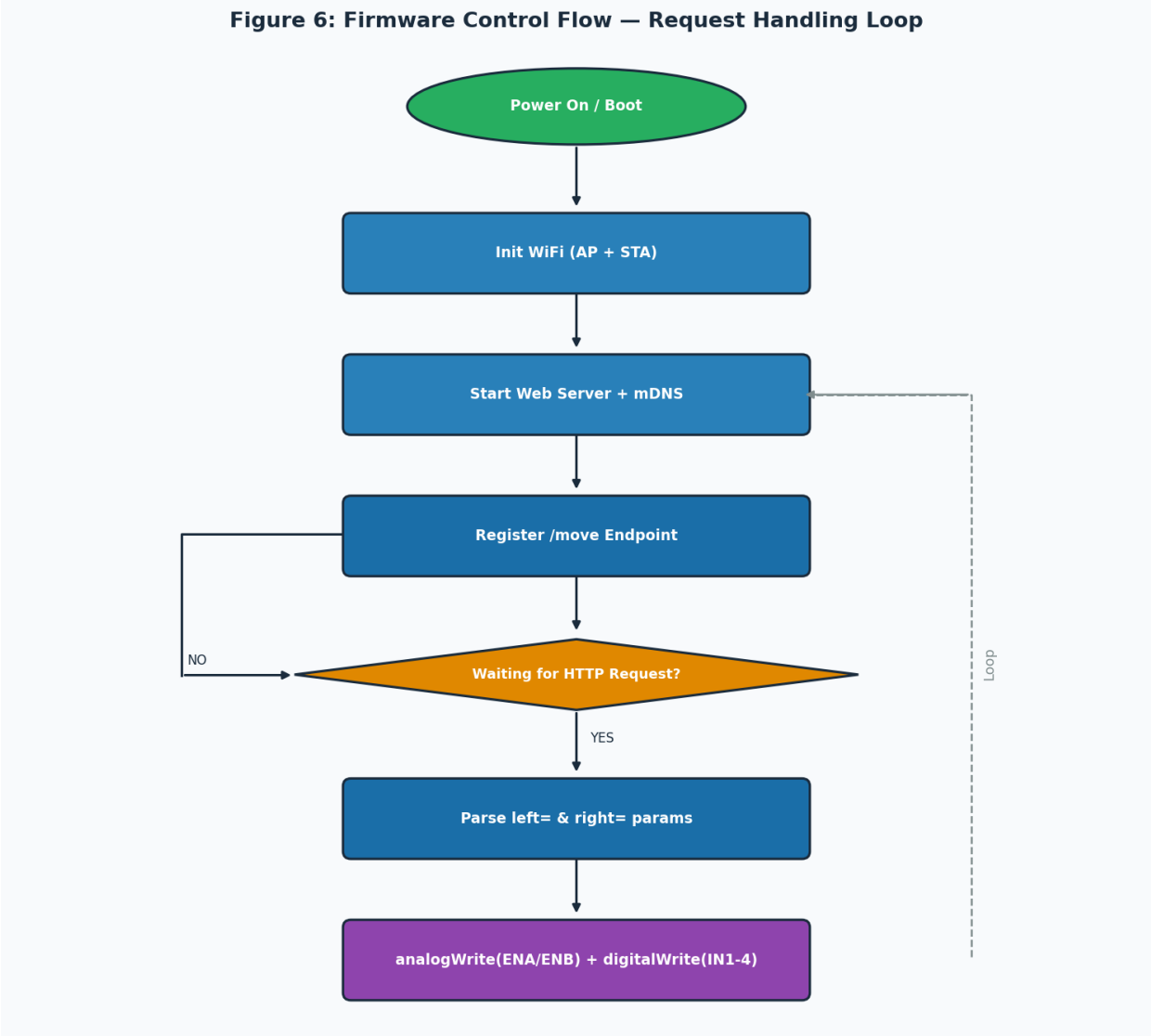

Figure 6: Firmware Control Flow — Boot to Motor Command Execution

5.1 The /move REST Endpoint

All motor control passes through a single HTTP GET endpoint:

```
GET http://<ROVER_IP>/move?left=<val>&right=<val>

Parameters:
  left  : integer, -1023 to +1023  (negative = reverse)
  right : integer, -1023 to +1023  (negative = reverse)

Example — full speed forward:
  GET /move?left=1023&right=1023
```

```
Example — gentle left turn:
  GET /move?left=200&right=800

Example — spin in place:
  GET /move?left=-1023&right=1023
```

The signed integer range maps directly to Arduino's analogWrite() range (0–255 for 8-bit PWM) via a linear scaling factor, and the sign controls the IN1/IN2 (or IN3/IN4) direction bits. This clean separation of speed and direction into a single signed value simplifies both the firmware and any client code.

5.2 Technology Stack

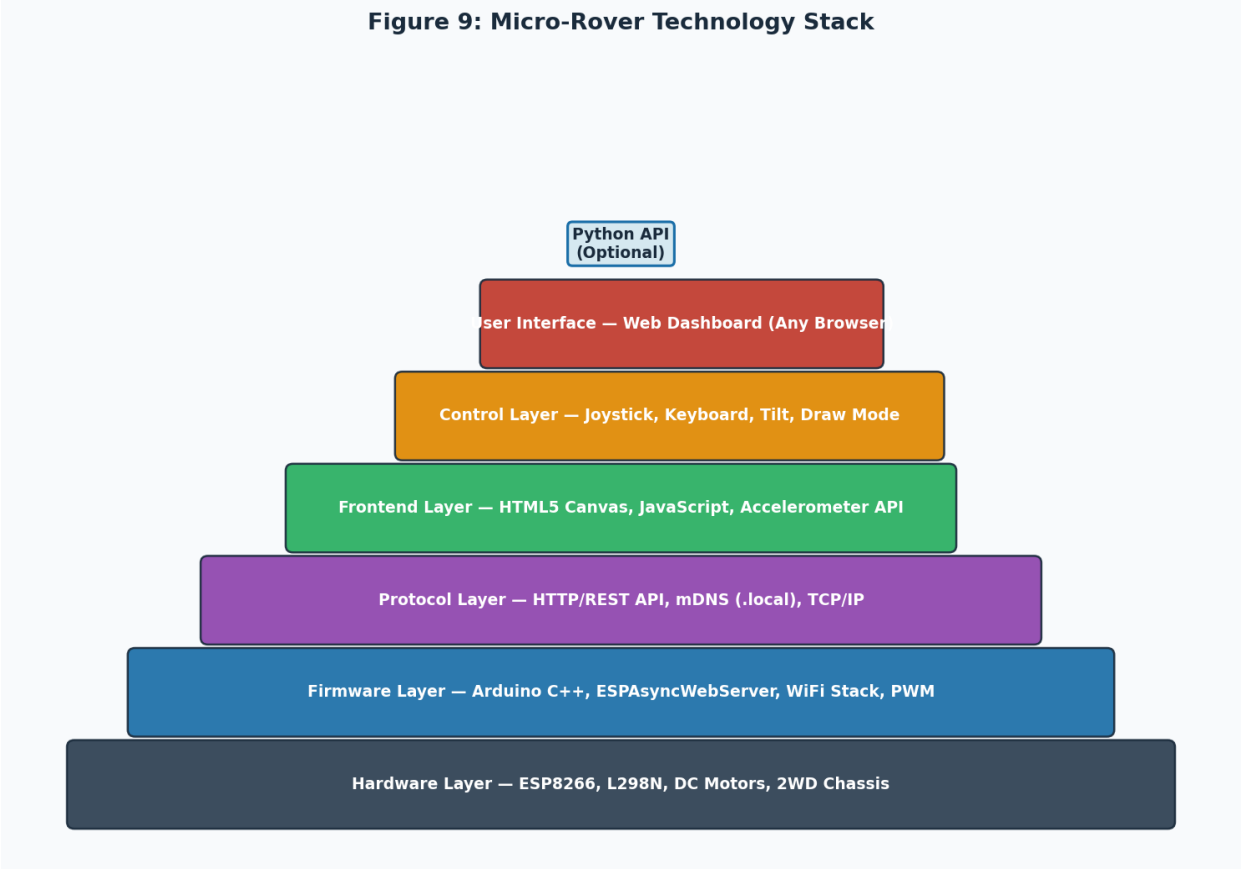

Figure 9: Micro-Rover Technology Stack — Layer Architecture

The stack spans six layers from bare silicon to end-user interface. Each layer has a clearly defined responsibility and communicates with adjacent layers through well-defined interfaces: GPIO for hardware-to-firmware, HTTP/REST for firmware-to-frontend, and DOM events for frontend-to-user. This separation of concerns makes the codebase maintainable and each layer independently testable.

6. Control Modalities

6.1 Virtual Joystick

The virtual joystick is implemented as an HTML5 Canvas element. Touch/mouse events are captured and converted to a polar coordinate (angle θ, magnitude r). These are then mapped to differential motor speeds using:

```
// Joystick polar-to-differential mapping (pseudocode)
left_speed  = r * cos(θ - π/4)   // ≈ forward + left component
right_speed = r * cos(θ + π/4)   // ≈ forward + right component
// Scale to ±1023 and send via HTTP
```

6.2 Keyboard Control

WASD and arrow key events are captured with keydown/keyup listeners. Each key combination maps to a predefined left/right motor speed pair: forward (1023, 1023), backward (-1023, -1023), left (0, 1023), right (1023, 0). Release events send a stop command (0, 0). Latency is limited primarily by the HTTP round-trip time, typically 35–50ms on a local network.

6.3 Tilt Control (Accelerometer Steering)

The DeviceOrientation API (available in all modern mobile browsers) provides the device's alpha, beta, and gamma Euler angles in real time. The firmware maps beta (front-back tilt) to forward/backward speed and gamma (side tilt) to differential steering. No additional hardware is required — the phone's built-in MEMS accelerometer is the controller.

6.4 Draw Mode — Path Following

Draw Mode is the most innovative feature of this project. Its operation is as follows:

1. User taps "Draw Mode: ON". The joystick area enters recording mode.
2. Every joystick movement generates a timed motor command pair {left, right, duration_ms}.
3. The drawn trajectory is rendered in yellow on the canvas in real time.
4. User taps "Follow Path". The recorded command sequence is replayed to the rover with identical timing.

This is an open-loop playback approach — it does not use wheel encoders or feedback control. Accuracy depends on surface friction and battery voltage consistency. On smooth, flat surfaces with a fully charged power bank, repeatability is reliable for paths up to approximately 3–4 metres.

7. Differential Drive Kinematics

Understanding differential drive kinematics is essential for interpreting the trajectory canvas and designing autonomous paths. Figure 7 visualises three fundamental steering modes.

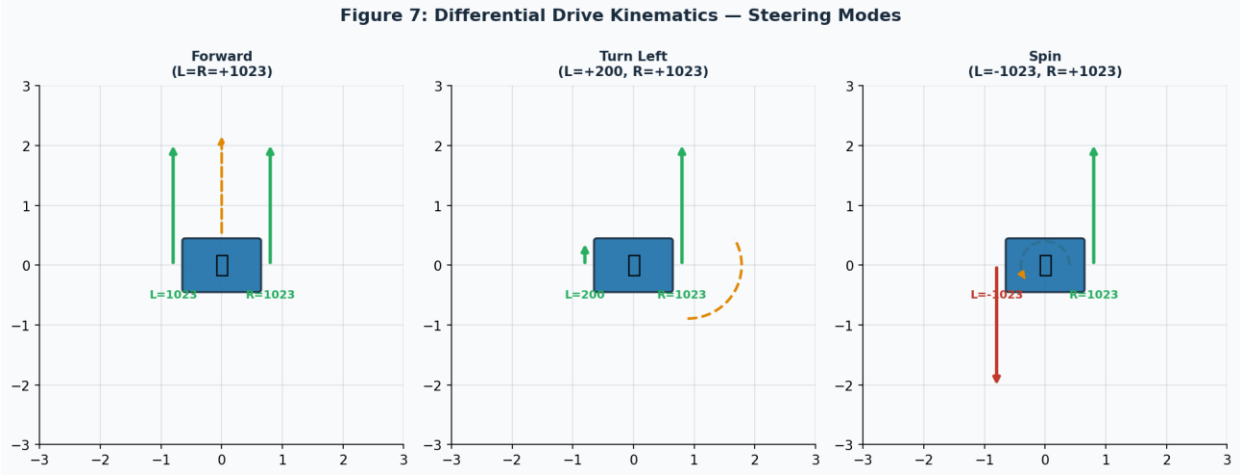

Figure 7: Differential Drive Kinematics — Forward, Curved Turn, and In-Place Spin

For a rover with wheel separation L (metres) and wheel radius r (metres), the instantaneous linear velocity v and angular velocity ω are:

```
v = r × (ω_R + ω_L) / 2        // average linear velocity
ω = r × (ω_R - ω_L) / L        // angular velocity

Turning radius R_turn = (L / 2) × (ω_R + ω_L) / (ω_R - ω_L)

Special cases:
  ω_L = ω_R   →  straight line (R_turn = ∞)
  ω_L = -ω_R  →  spin in place (R_turn = 0)
  ω_L = 0     →  pivot on left wheel
```

These relationships are used by the trajectory canvas to simulate the rover's position. Given the PWM values sent to each motor, the canvas estimates relative wheel speeds, integrates the kinematic equations over time, and renders the path on-screen.

8. Power Consumption Analysis

Figures 4 and 5 provide a comprehensive power budget and runtime projection for the Micro-Rover platform across different operating conditions and power bank sizes.

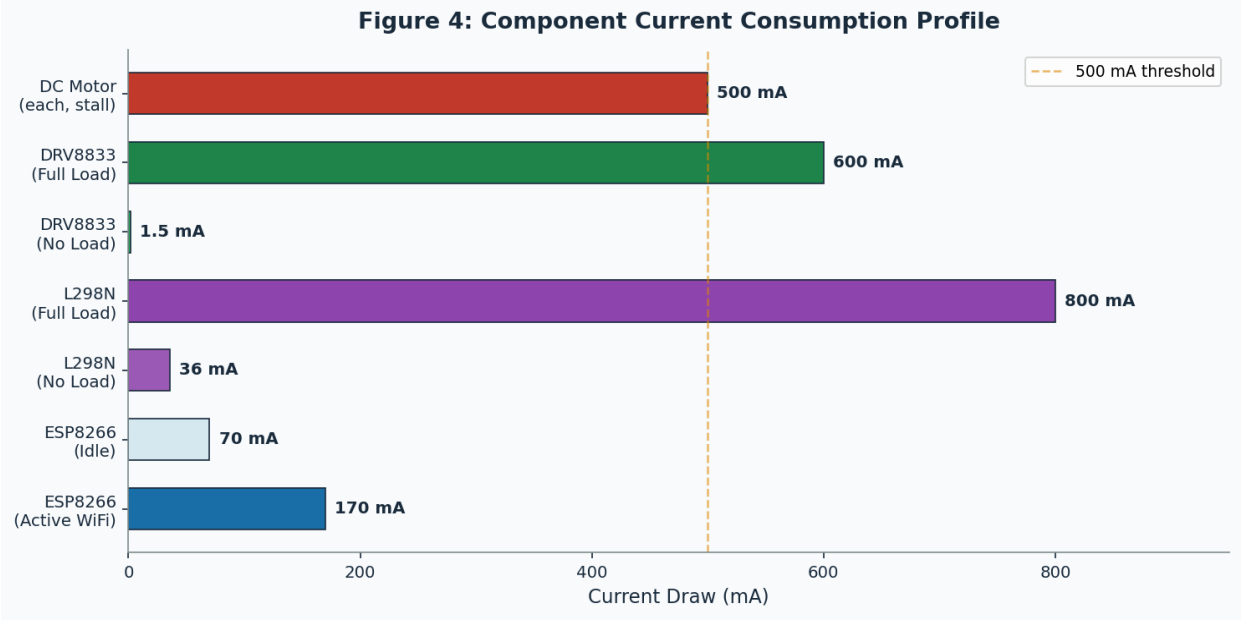

Figure 4: Component Current Consumption Profile (mA)

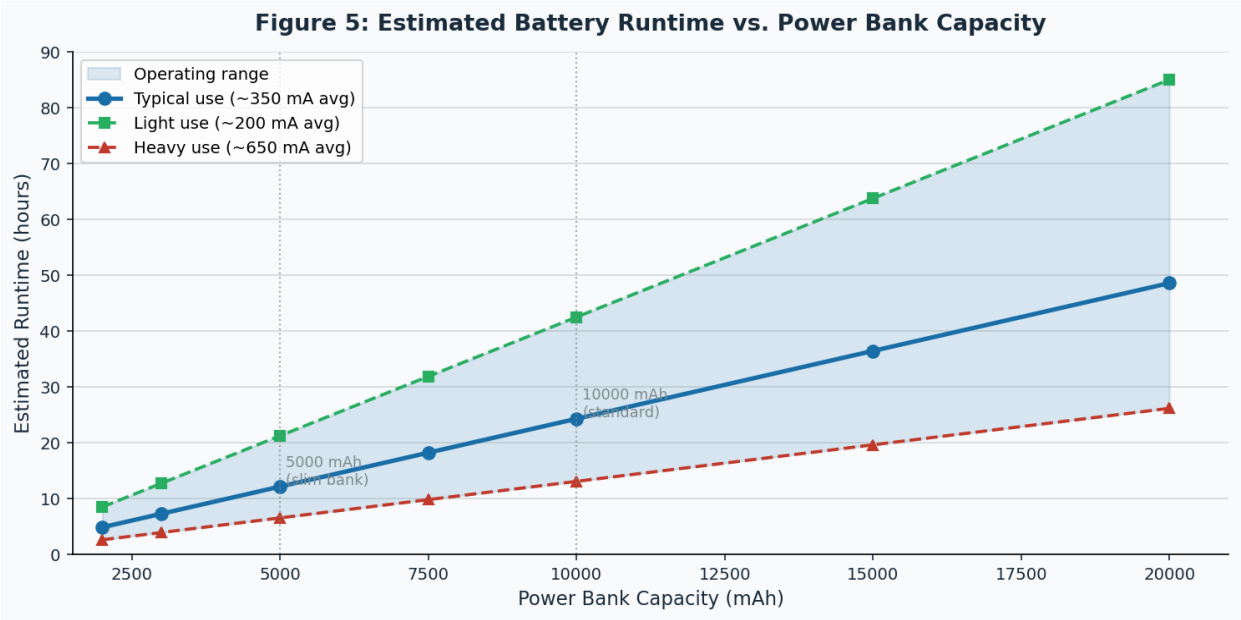

Figure 5: Estimated Battery Runtime vs. Power Bank Capacity

8.1 Power Budget

Table 3: System Power Budget

Component	Idle (mA)	Active (mA)	Peak (mA)	Notes
ESP8266 (WiFi AP)	70	170	350	Transmit bursts up to 350mA
L298N (no motors)	36	36	36	Quiescent draw
L298N (motors)	0	400	800	Both motors mid-load
DRV8833 (motors)	0	250	600	More efficient than L298N
Total (L298N)	106	606	1150	Both motors half-load + WiFi
Total (DRV8833)	70	420	950	Recommended for battery life

8.2 Runtime Estimation

Usable energy from a power bank is approximately 85% of rated capacity due to conversion losses from the internal lithium cell (~3.7V) to the output 5V USB rail. Runtime is estimated as:

```
Runtime (h) = Capacity_mAh × 0.85 / Average_Current_mA

Example: 10,000 mAh bank, typical mixed use (~350 mA avg):
Runtime = 10000 × 0.85 / 350 ≈ 24.3 hours

Example: 5,000 mAh bank, heavy motor use (~650 mA avg):
Runtime = 5000 × 0.85 / 650 ≈ 6.5 hours
```

Power Bank Selection Tips:
- Use a bank that supports continuous low-current output. Some banks auto-shutoff below ~100mA.
- The DRV8833 motor driver is recommended over L298N for ~30-40% better battery efficiency.
- A 5000 mAh bank provides 3–5 hours typical operation — more than sufficient for demos.
- Two USB outputs are ideal: one for ESP8266, one for the motor driver.

9. Performance Analysis

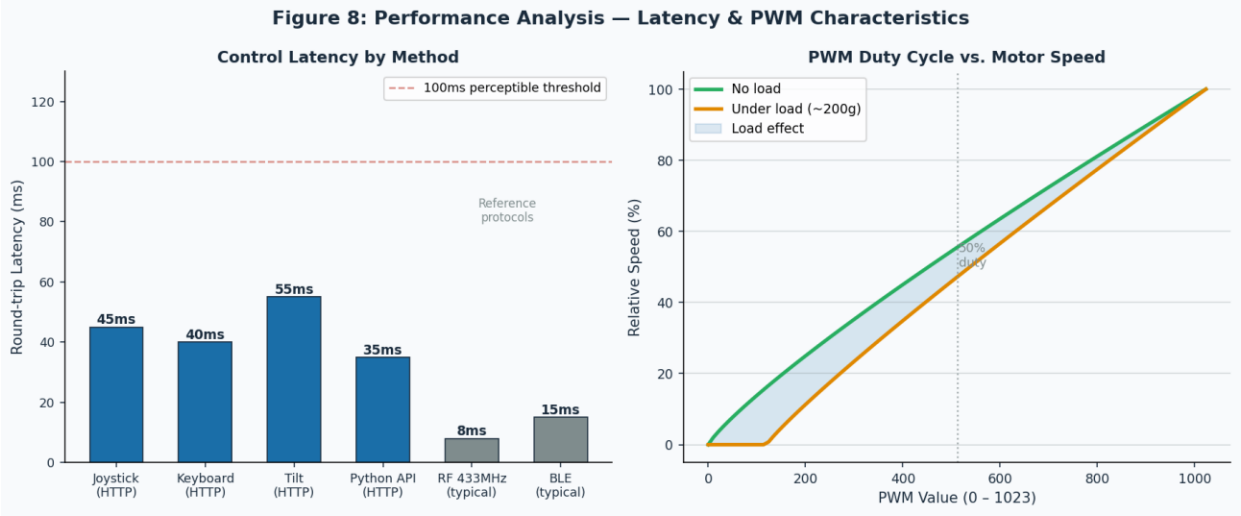

Figure 8: Latency Comparison (left) and PWM-to-Speed Characteristic (right)

9.1 Control Latency

HTTP-based control introduces round-trip latency of 35–55ms on a local WiFi network depending on control modality. This is above the latency of dedicated RF protocols (typically 8–15ms) but remains well below the 100ms threshold at which humans perceive control lag in driving tasks. For the rover's speed range (typically 0.3–0.8 m/s), a 50ms delay corresponds to a position error of 1.5–4 cm — acceptable for most use cases.

9.2 PWM Speed Control

The right panel of Figure 8 shows the nonlinear relationship between PWM duty cycle and motor speed. DC motors exhibit a dead band at low PWM values where torque is insufficient to overcome static friction. This dead band is larger under load. In practice, PWM values below approximately 150–200 (out of 1023) may not produce movement on typical surfaces. The joystick and keyboard control code should incorporate a minimum threshold to avoid sending commands in this dead band.

10. Software Setup & Deployment

10.1 Arduino IDE Configuration

5. Install Arduino IDE from arduino.cc/en/software
6. Add the ESP8266 board package URL in File > Preferences > Additional Board URLs:

```
http://arduino.esp8266.com/stable/package_esp8266com_index.json
```

7. Install the ESP8266 package via Tools > Board > Boards Manager
8. Clone the repository:

```
git clone https://github.com/RemanDey/micro-rover.git
```

9. Open main/main.ino and update WiFi credentials (optional):

```
#define WIFI_STA_SSID  "YourNetworkName"
#define WIFI_STA_PASS  "YourPassword"
```

10. Select board: Tools > Board > NodeMCU 1.0 (ESP-12E) or Wemos D1 Mini
11. Click Upload. The ESP8266 reboots and begins serving the dashboard.

10.2 Python API for Automation

The open REST API enables programmatic control from any Python environment — a laptop, Raspberry Pi, or automated test script:

```
import requests

ROVER = 'http://micro-rover.local'   # or IP address

def move(left, right):
    requests.get(f'{ROVER}/move?left={left}&right={right}', timeout=0.5)

# Drive forward at full speed for 2 seconds
import time
move(1023, 1023)
time.sleep(2)
move(0, 0)       # stop

# Spin right 90 degrees (approx)
move(1023, -1023)
time.sleep(0.35) # tune for your rover
move(0, 0)
```

11. Troubleshooting Guide

Table 4: Common Issues & Solutions

Symptom	Likely Cause	Solution
One motor wrong direction	Wires swapped	Swap motor wire pair at driver terminal, or swap IN1↔IN2 in code
Can't reach micro-rover.local	mDNS not resolving	Use IP from Serial Monitor at 115200 baud instead
Power bank shuts off	Low-current auto-shutoff	Add 100Ω resistor + LED as dummy load, or use 'always-on' bank
Rover drifts in straight line	Motor mismatch	Trim ENA/ENB PWM values to balance motor speeds
Web page won't load	Wrong IP / not connected	Reconnect to micro_rover WiFi, navigate to 192.168.4.1

Tilt control unresponsive	Browser permission denied	Enable motion sensors in browser settings; iOS needs explicit permission
Draw Mode inaccurate	Open-loop, no encoders	Use smooth flat surface; reduce max speed for better repeatability
Upload fails	Wrong board/port selected	Select NodeMCU 1.0 or Wemos D1 Mini; check COM port in Device Manager

12. Future Work & Extension Possibilities

The Micro-Rover platform is deliberately minimal, making it an excellent base for further experimentation:

- Wheel Encoders: Adding quadrature encoders enables closed-loop speed control and accurate odometry, greatly improving Draw Mode path accuracy.
- Obstacle Avoidance: An HC-SR04 ultrasonic sensor on the front can be read via a second async endpoint, enabling autonomous stopping or steering.
- WebSocket Control: Replacing HTTP GET with a persistent WebSocket connection would reduce latency from ~45ms to ~5ms, enabling faster, more responsive steering.
- Camera Streaming: The ESP32-CAM (ESP8266 successor with camera support) could provide a live FPV feed streamed alongside the control dashboard.
- PID Speed Control: With encoders, a PID loop on each motor would compensate for battery voltage sag during runtime, maintaining consistent speed.
- MQTT Integration: Publishing sensor data and subscribing to commands via MQTT would allow the rover to integrate with home automation platforms like Home Assistant.

13. Conclusion

Micro-Rover (Zade) demonstrates that sophisticated mobile robotics — multiple control modalities, real-time trajectory visualization, autonomous path following, and a programmable REST API — can be achieved on a $10–15 hardware budget powered entirely by a standard USB power bank. The ESP8266's integrated WiFi and capable processor eliminate the need for separate radio modules or companion computers, and its web server capability means any device with a browser can serve as the control console.

The project is fully open-source under the MIT License, documented here for reproducibility, and available at github.com/RemanDey/micro-rover. It is submitted to the Instructables Battery-Powered Contest as evidence that battery-only operation does not imply capability compromise — it imposes a design discipline that results in leaner, more efficient, and ultimately more elegant embedded systems.

Repository: github.com/RemanDey/micro-rover | License: MIT

References

[1] ESP8266 Technical Reference, Espressif Systems, v1.7, 2020.

[2] Siegwart, R., Nourbakhsh, I.R., & Scaramuzza, D. (2011). Introduction to Autonomous Mobile Robots (2nd ed.). MIT Press.

[3] ESPAsyncWebServer Library Documentation. https://github.com/ESP8266/Arduino

[4] HTML Living Standard — DeviceOrientation Event Specification. WHATWG, 2024.

[5] M. Quigley et al., "ROS: an open-source Robot Operating System," ICRA Workshop on Open Source Software, 2009.

[6] Micro-Rover Source Repository. https://github.com/RemanDey/micro-rover (MIT License)